code

October 19, 2026

[70]:
```
import numpy as np
import pandas as pd
```

[71]:
```
#Q1.(a)
problem1 = pd.read_csv('problem1.csv')

from Risk_Library import covariance_matrix

cor_matrix = covariance_matrix(problem1, method= "correlation",␣
 ↪missing_strategy= "pairwise")
print("pairwise correlation matrix:")
print(cor_matrix)
```

```
pairwise correlation matrix:
          x1        x2        x3        x4        x5
x1  1.000000  0.332836  0.418462  0.255285  0.732272
x2  0.332836  1.000000  0.251035  0.287709  0.787538
x3  0.418462  0.251035  1.000000  0.483580  0.684257
x4  0.255285  0.287709  0.483580  1.000000  0.782632
x5  0.732272  0.787538  0.684257  0.782632  1.000000
```

[72]:
```
#Q1.(b)

from Risk_Library import first_four_moments

f4m = first_four_moments(problem1)
print(f4m)
print("")
print("So, the variance of each series x1 - x5:")
print(f4m.loc["Variance", "x1"])
print(f4m.loc["Variance", "x2"])
print(f4m.loc["Variance", "x3"])
print(f4m.loc["Variance", "x4"])
print(f4m.loc["Variance", "x5"])
```

```
                x1        x2        x3        x4        x5
Mean      0.009953  0.009416 -0.000599 -0.006344  0.002963
Variance  0.000200  0.000286  0.000178  0.000624  0.000134
Skewness  0.679943 -0.154428  0.721462  0.536777  0.038895
```

```
Kurtosis  1.438082 -0.742252  0.875989  0.560214  0.078110

So, the variance of each series x1 - x5:
0.00019998525040998153
0.0002857520761775105
0.0001777813494044681
0.0006235346985142352
0.00013433447338765598
```

[73]:
```
#Q1.(c)

from Risk_Library import classify_psd

cls_results = classify_psd(cor_matrix)
print("Eigenvalues:")
print(cls_results["eigenvalues"])
print("")

print("From the eigenvalues shown above we noticed that there is one values␣
 ↪that is significantly < 0")
print("So the correlation matrix is non-PSD")
```

```
Eigenvalues:
[-0.08927867  0.4613398   0.71908517  0.82343722  3.08541647]

From the eigenvalues shown above we noticed that there is one values that is
significantly < 0
So the correlation matrix is non-PSD
```

[74]:
```
#Q1.(d)

from Risk_Library import fix_psd_matrix

fix_cor_matrix = fix_psd_matrix(cor_matrix, method= "higham")
print("Fixed matrix:")
print(fix_cor_matrix)
print("")

new_cl = classify_psd(fix_cor_matrix)
print("Eigenvalue:")
print(new_cl["eigenvalues"])
print("")


diff = np.abs(cor_matrix - fix_cor_matrix)
max_change = np.max(diff)
print("Maximum change:")
print(max_change)
```

```
max_change_indices = np.where(diff == max_change)
print("Indices of maximum change")
print(max_change_indices)
```

```
Fixed matrix:
          x1        x2        x3        x4        x5
x1  1.000000  0.349452  0.425942  0.270782  0.694572
x2  0.349452  1.000000  0.260599  0.307523  0.739337
x3  0.425942  0.260599  1.000000  0.492500  0.662558
x4  0.270782  0.307523  0.492500  1.000000  0.737677
x5  0.694572  0.739337  0.662558  0.737677  1.000000

Eigenvalue:
[-5.29177212e-10  4.50081533e-01  7.03476106e-01  8.06708468e-01
  3.03973389e+00]

Maximum change:
0.04820114199011649
Indices of maximum change
(array([1, 4]), array([4, 1]))
```

[75]:
```
#Q1.(e)
cov_matrix = covariance_matrix(problem1, method= "covariance",␣
 ↪missing_strategy= "pairwise")
print(cov_matrix)
```

```
          x1        x2        x3        x4        x5
x1  0.000406  0.000153  0.000101  0.000128  0.000220
x2  0.000153  0.000480  0.000074  0.000172  0.000273
x3  0.000101  0.000074  0.000151  0.000181  0.000148
x4  0.000128  0.000172  0.000181  0.000842  0.000356
x5  0.000220  0.000273  0.000148  0.000356  0.000273
```

[76]:
```
#Q2.(a,b)
from scipy import stats
from Risk_Library import return_calculate, fit_normal_distribution,␣
 ↪calculate_var, calculate_es

problem2 = pd.read_csv("problem2.csv")
returns_2 = return_calculate(problem2, method="arithmetic", date_column="Date")
r_spy = returns_2["SPY"].values
r_xyz = returns_2["XYZ"].values
n2 = len(r_xyz)

# OLS regression of XYZ on SPY (inline: Library has only a t-error regression,␣
 ↪fit_t_regression)
X = np.column_stack([np.ones(n2), r_spy])
```

```
(alpha_hat, beta_hat), *_ = np.linalg.lstsq(X, r_xyz, rcond=None)
residuals = r_xyz - (alpha_hat + beta_hat * r_spy)
sigma_eps = residuals.std(ddof=2)

spy_fit = fit_normal_distribution(r_spy)
mu_spy = spy_fit["mu"]
sigma_spy = spy_fit["sigma"]

# Distribution of r_XYZ implied by the equation: a linear combination of two␣
 ↪independent normals is normal
# mean = alpha + beta * mu_SPY,  variance = beta^2 * sigma_SPY^2 + sigma_eps^2
mu_xyz = alpha_hat + beta_hat * mu_spy
sigma_xyz = np.sqrt(beta_hat**2 * sigma_spy**2 + sigma_eps**2)

print("Fitted parameters:")
print(pd.Series({"alpha": alpha_hat, "beta": beta_hat, "sigma_eps": sigma_eps,
                 "mu_SPY": mu_spy, "sigma_SPY": sigma_spy,
                 "mu_XYZ (implied)": mu_xyz, "sigma_XYZ (implied)": sigma_xyz}).
 ↪to_string())
print("")


def normal_var_es(mu, sigma, alpha):
    z = stats.norm.ppf(alpha)
    return -(mu + sigma * z), -(mu - sigma * stats.norm.pdf(z) / alpha)


rng2 = np.random.default_rng(545)
N_SIM2 = 5_000
sim_xyz = (alpha_hat + beta_hat * rng2.normal(mu_spy, sigma_spy, N_SIM2)
           + rng2.normal(0.0, sigma_eps, N_SIM2))

rows2 = {}
for level, alpha in [("95%", 0.05), ("99%", 0.01)]:
    var_a, es_a = normal_var_es(mu_xyz, sigma_xyz, alpha)
    rows2[level] = {
        "VaR (%)": var_a * 100,
        "ES (%)": es_a * 100,
        "VaR (%) simulation check": calculate_var(sim_xyz, method="historical",␣
 ↪alpha=alpha)["VaR Absolute"] * 100,
        "ES (%) simulation check": calculate_es(sim_xyz, method="historical",␣
 ↪alpha=alpha)["ES Absolute"] * 100,
    }
q2_equation_table = pd.DataFrame(rows2).T
q2_equation_table
```

```
Fitted parameters:
```

```
alpha                 -3.040218e-04
beta                   1.193311e+00
sigma_eps              1.460469e-02
mu_SPY                 2.547717e-04
sigma_SPY              1.097207e-02
mu_XYZ (implied)       3.144186e-18
sigma_XYZ (implied)    1.961443e-02
```

[76]:

	VaR (%)	ES (%)	VaR (%) simulation check	ES (%) simulation check
95%	3.226287	4.045894	3.113469	3.897526
99%	4.563000	5.227667	4.536029	5.026366

0.1 Q2 Explanation

Here since we assumed XYZ is related to the market (SPY) by the following equation: = + + e

Then I regressed xyz on spy and modeled spy with a normal distribution to get the params and use that to calculate the distrbution parameter of the fitted xyz return.

From there i could build return of xyz from the equation and get the simulated historical results and take the quantiles 5% and 1% to check and calculate VaR and ES.

[ ]:
```python
#Q3.(a)
problem3_price = pd.read_csv("problem3.csv")
problem3_holdings = pd.read_csv("problem3_holdings.csv")
tickers = [c for c in problem3_price.columns if c != "Date"]

returns_10 = return_calculate(problem3_price, method="arithmetic",␣
 ↪date_column="Date")

from Risk_Library import fit_normal_distribution, fit_t_distribution,␣
 ↪t_distribution_aicc
from Risk_Library import fit_nig_distribution


from scipy import stats
from utils import aicc

def normal_aicc(data, params):
    # (inline: not in Library) AICc of a fitted normal = Library's utils.aicc␣
 ↪applied to the
    # normal log-likelihood with 2 parameters (mu, sigma). The t version is␣
 ↪t_distribution_aicc.
    loglik = stats.norm.logpdf(data, loc=params["mu"], scale=params["sigma"]).
 ↪sum()
    return aicc(loglik, num_params=2, num_obs=len(data))
```

```
def nig_aicc(data, nig_dist):
    # (inline: not in Library) AICc of a fitted NIG = Library's utils.aicc␣
 ↪applied to the NIG
    # log-likelihood with 4 parameters (mu, alpha, beta, delta). `nig_dist` is␣
 ↪the frozen
    # scipy.stats.norminvgauss returned by fit_nig_distribution(...,␣
 ↪output="both").
    loglik = nig_dist.logpdf(data).sum()
    return aicc(loglik, num_params=4, num_obs=len(data))


fit_rows = {}
best_dist10 = {}

for tk in tickers:
    x = returns_10[tk]

    n_fit = fit_normal_distribution(x)
    t_fit = fit_t_distribution(x)
    nig_fit, nig_dist = fit_nig_distribution(x, method="mle", output="both")

    aiccs = {
        "Normal": normal_aicc(x, n_fit),
        "T": t_distribution_aicc(x, fitted=t_fit),
        "NIG": nig_aicc(x, nig_dist),
    }
    best = min(aiccs, key=aiccs.get)   # name of the distribution with the␣
 ↪lowest AICc
    best_dist10[tk] = best

    fit_rows[tk] = {
        "Normal mu": n_fit["mu"],
        "Normal sigma": n_fit["sigma"],
        "Normal AICc": aiccs["Normal"],

        "T mu": t_fit["mu"],
        "T sigma": t_fit["sigma"],
        "T nu": t_fit["nu"],
        "T AICc": aiccs["T"],

        "NIG mu": nig_fit["mu"],
        "NIG alpha": nig_fit["alpha"],
        "NIG beta": nig_fit["beta"],
        "NIG delta": nig_fit["delta"],
        "NIG AICc": aiccs["NIG"],

        "Best fit": best,
```

```
    }
fits10 = pd.DataFrame(fit_rows).T
fits10
```

[ ]:

	Normal mu	Normal sigma	Normal AICc	T mu	T sigma	T nu	\
S1	0.000904	0.011826	-6033.993247	0.000904	0.01182	355368542.589316	
S2	0.0008	0.015348	-5512.705402	0.000581	0.011673	4.857877	
S3	0.000424	0.018898	-5096.512392	0.002291	0.013657	4.160482	
S4	0.00078	0.009933	-6382.973454	0.00078	0.009928	339036291.501084	
S5	0.000709	0.018951	-5090.894707	0.000833	0.015577	6.289248	
S6	0.000805	0.015085	-5547.202967	0.002445	0.010655	3.823756	
S7	0.000693	0.013674	-5743.69378	0.000692	0.013668	319103.362911	
S8	0.000374	0.017112	-5295.106437	0.000654	0.011424	3.483289	
S9	0.001746	0.020647	-4919.505663	0.003383	0.01531	4.385214	
S10	0.000165	0.013061	-5835.324216	0.000405	0.009427	4.12127	

	T AICc	NIG mu	NIG alpha	NIG beta	NIG delta	NIG AICc	\
S1	-6031.981687	0.076557	2086.4856	-588.204192	0.257473	-6030.21506	
S2	-5627.359694	-0.000305	73.206598	4.863823	0.016593	-5622.063046	
S3	-5238.463137	0.010474	79.836689	-36.344624	0.019657	-5281.640956	
S4	-6380.961893	0.089549	3368.189374	-984.957946	0.290287	-6379.119707	
S5	-5150.869475	0.001497	73.576752	-2.232562	0.025958	-5146.209539	
S6	-5688.740013	0.008272	91.682513	-41.686207	0.014628	-5737.395453	
S7	-5741.682006	0.077482	1635.963518	-443.712607	0.272611	-5739.887758	
S8	-5493.287554	0.001183	50.62284	-2.886825	0.014161	-5487.094112	
S9	-5036.762452	0.010128	63.159483	-22.926741	0.021515	-5065.509637	
S10	-5974.85701	0.001049	76.685731	-5.368403	0.012595	-5970.701785	

	Best fit
S1	Normal
S2	T
S3	NIG
S4	Normal
S5	T
S6	NIG
S7	Normal
S8	T
S9	NIG
S10	T

0.2 Q3.(a) Explaination

The best fitted distributions are: x1 - Normal x2 - T x3 - NIG x4 - Normal x5 - T x6 - NIG x7 - Normal x8 - T x9 - NIG x10 - T

And each selection is made by looking at the fitted AICc score. For each stock, the fitted distribution with the lowest AICc score is the best distiburion.

[ ]:
```
#Q3.(b)
from Risk_Library import kendall_correlation_matrix, compare_copulas
from copula import _fit_t_copula_nu

returns_only = returns_10.set_index("Date")[tickers]   # drop the Date column,␣
 ↪keep the 10 return series
n_obs = len(returns_only)

# 1. Frozen scipy distribution for each stock's chosen marginal from part (a)␣
 ↪(best_dist10)
def fit_chosen_marginal(x, dist_name):
    if dist_name == "Normal":
        return fit_normal_distribution(x, output="distribution")
    if dist_name == "T":
        return fit_t_distribution(x, output="distribution")
    if dist_name == "NIG":
        return fit_nig_distribution(x, method="mle", output="distribution")
    raise ValueError(f"Unknown distribution {dist_name}")

marginals10 = {tk: fit_chosen_marginal(returns_only[tk], best_dist10[tk]) for␣
 ↪tk in tickers}

# 2. Probability integral transform: U = F(x) with each stock's own fitted CDF.
#    Clipped away from exactly 0/1 so the inverse CDFs below stay finite.
U = np.column_stack([marginals10[tk].cdf(returns_only[tk].values) for tk in␣
 ↪tickers])
U = np.clip(U, 1e-12, 1 - 1e-12)

# 3. Copula correlation: sin(pi/2 * Kendall's tau), PSD-repaired (same matrix␣
 ↪compare_copulas uses
#    for both copulas). Kendall's tau is rank-based, so it doesn't depend on␣
 ↪the marginals.
copula_corr = kendall_correlation_matrix(returns_only)
R = copula_corr.values
R_inv = np.linalg.inv(R)
_, logdet_R = np.linalg.slogdet(R)

# 4a. Gaussian copula log-likelihood: z = Phi^-1(U),
#     log c(u) = -0.5*log|R| - 0.5*(z' R^-1 z - z' z), summed over all days
z = stats.norm.ppf(U)
gauss_ll = np.sum(-0.5 * logdet_R
                  - 0.5 * (np.einsum("ij,jk,ik->i", z, R_inv, z) - np.
 ↪einsum("ij,ij->i", z, z)))

# 4b. t copula: same correlation, fit the copula's degrees of freedom nu by␣
 ↪maximum likelihood
```

```
t_copula_nu, t_ll = _fit_t_copula_nu(U, R)

# 5. Compare (same layout as compare_copulas): K = parameters beyond the shared␣
 ↪correlation matrix
copula_compare = pd.DataFrame({
    "Gaussian": {"LL": gauss_ll, "K": 0, "AICc": aicc(gauss_ll, 0, n_obs),␣
 ↪"BIC": -2 * gauss_ll},
    "T":        {"LL": t_ll, "K": 1, "AICc": aicc(t_ll, 1, n_obs), "BIC": -2 *␣
 ↪t_ll + 1 * np.log(n_obs)},
}).T
print("Marginals used:", best_dist10)
print("t copula nu:", t_copula_nu)
print("\nCopula correlation matrix:")
print(copula_corr.round(4).to_string())
copula_compare
```

```
Marginals used: {'S1': 'Normal', 'S2': 'T', 'S3': 'NIG', 'S4': 'Normal', 'S5':
'T', 'S6': 'NIG', 'S7': 'Normal', 'S8': 'T', 'S9': 'NIG', 'S10': 'T'}
t copula nu: 999.9998212132779

Copula correlation matrix:
         S1      S2      S3      S4      S5      S6      S7      S8      S9
S10
S1   1.0000  0.5430  0.5282  0.5618  0.5420  0.2941  0.2487  0.2315  0.2811
0.2930
S2   0.5430  1.0000  0.5637  0.5301  0.5232  0.3296  0.2331  0.2461  0.3104
0.2762
S3   0.5282  0.5637  1.0000  0.5320  0.5706  0.3163  0.3026  0.2678  0.3400
0.2983
S4   0.5618  0.5301  0.5320  1.0000  0.5682  0.3060  0.2328  0.2686  0.2962
0.2991
S5   0.5420  0.5232  0.5706  0.5682  1.0000  0.3331  0.2699  0.2794  0.2763
0.2959
S6   0.2941  0.3296  0.3163  0.3060  0.3331  1.0000  0.5346  0.5244  0.5391
0.5467
S7   0.2487  0.2331  0.3026  0.2328  0.2699  0.5346  1.0000  0.5258  0.5223
0.5421
S8   0.2315  0.2461  0.2678  0.2686  0.2794  0.5244  0.5258  1.0000  0.5204
0.5240
S9   0.2811  0.3104  0.3400  0.2962  0.2763  0.5391  0.5223  0.5204  1.0000
0.5217
S10  0.2930  0.2762  0.2983  0.2991  0.2959  0.5467  0.5421  0.5240  0.5217
1.0000
```

[ ]:

	LL	K	AICc	BIC
Gaussian	2110.022340	0.0	-4220.044679	-4220.044679
T	2110.021913	1.0	-4218.039819	-4213.136071

0.3 Q3.(b)

Based on the table shown in the previous question. T copula is using one extra more parameter then that in the Gaussian copula and both AICc and BICc scores are lower in Gaussian copula. So will make a selection of Gaussian copula here.

[ ]:
```
#Q3.(c)
from copula import _simulate_copula_uniforms


CHOSEN_COPULA = "gaussian"
NUM_SIMS = 10_000
rng = np.random.default_rng(545)

# 1. Simulate correlated uniforms from the copula -> shape (num_stocks,␣
 ↪NUM_SIMS)
sim_U = _simulate_copula_uniforms(
    copula_corr.values, CHOSEN_COPULA,
    t_copula_nu if CHOSEN_COPULA == "t" else None,
    NUM_SIMS, rng,
)

# 2. Map each stock's uniforms back to returns with its own chosen marginal␣
 ↪from part (a): r = F^-1(U)
sim_returns = pd.DataFrame({tk: marginals10[tk].ppf(sim_U[i]) for i, tk in␣
 ↪enumerate(tickers)})

# 3. Position values at the last price in the file
holdings = problem3_holdings.set_index("Stock").loc[tickers]
last_prices = problem3_price[tickers].iloc[-1]
position_value = holdings["Holding"] * last_prices

# 4. Simulated $ P&L: each stock = position value * simulated return; books and␣
 ↪total = sums of their stocks
sim_pnl = sim_returns * position_value
for book in sorted(holdings["Book"].unique()):
    book_stocks = holdings.index[holdings["Book"] == book]
    sim_pnl[f"Book {book}"] = sim_pnl[book_stocks].sum(axis=1)
sim_pnl["Total"] = sim_pnl[tickers].sum(axis=1)

value_of = dict(position_value)
for book in sorted(holdings["Book"].unique()):
    value_of[f"Book {book}"] = position_value[holdings["Book"] == book].sum()
value_of["Total"] = position_value.sum()

# 5. 95% absolute VaR and ES in $ of each simulated P&L column. Library␣
 ↪calculate_var/es with
```

```
#    method="historical" = empirical 5% quantile / mean of the tail of the␣
 ↪sample it is given,
#    reported as positive losses measured from 0 ("Absolute").
ALPHA = 0.05
risk_rows = {}
for col in sim_pnl.columns:
    risk_rows[col] = {
        "Book": holdings.loc[col, "Book"] if col in tickers else "",
        "Current Value ($)": value_of[col],
        "VaR95 ($)": calculate_var(sim_pnl[col], method="historical",␣
 ↪alpha=ALPHA)["VaR Absolute"],
        "ES95 ($)": calculate_es(sim_pnl[col], method="historical",␣
 ↪alpha=ALPHA)["ES Absolute"],
    }
risk_table = pd.DataFrame(risk_rows).T
risk_table["VaR95 (%)"] = risk_table["VaR95 ($)"] / risk_table["Current Value␣
 ↪($)"]
risk_table["ES95 (%)"] = risk_table["ES95 ($)"] / risk_table["Current Value␣
 ↪($)"]
print(f"Copula: {CHOSEN_COPULA}, draws: {NUM_SIMS:,}")
risk_table
```

```
Copula: gaussian, draws: 10,000
```

[ ]:

	Book	Current Value ($)	VaR95 ($)	ES95 ($)	VaR95 (%)	ES95 (%)
S1	A	48329.478358	868.351679	1122.191054	0.017967	0.02322
S2	A	46697.425991	1093.264846	1621.564466	0.023412	0.034725
S3	A	24267.27694	805.00777	1209.510857	0.033173	0.049841
S4	A	43596.810952	672.127486	845.802191	0.015417	0.019401
S5	A	37727.450468	1072.918912	1534.368601	0.028439	0.04067
S6	B	46326.642687	1214.601127	1829.981086	0.026218	0.039502
S7	B	40144.306138	867.818596	1104.87645	0.021617	0.027523
S8	B	26045.880281	653.767193	997.495152	0.025101	0.038298
S9	B	98248.712042	3400.523489	5082.335955	0.034611	0.051729
S10	B	24512.380167	469.641033	742.702851	0.019159	0.030299
Book A		200618.442709	3597.61787	4943.928265	0.017933	0.024643
Book B		235277.921315	5451.210509	7856.422644	0.023169	0.033392
Total		435896.364024	7679.534187	10671.757826	0.017618	0.024482

0.4 Q3.(c) Discussion

Comparing book A we can tell that the with a small amount of larger value in total holding values the VaR is much larger that that in the Book B. So that means the book A is likly to be more risky in terms of baring a higher loss in bad days among the normal trading days we have here in the data.